

problem1

May 20, 2026

0.1 3.1 Markovian drone

(a)

```
[1]: import numpy as np
import matplotlib.pyplot as plt

n=20
drone_pos=np.array((0,0))
goal_pos=np.array((19,9))
x_eye=np.array((15,15))
sigma=10
gamma=0.95
Value_function=np.zeros((20,20))
previous_value_function=np.zeros((20,20))

actions={"up": np.array((0,1)),
         "down": np.array((0,-1)),
         "left": np.array((-1,0)),
         "right": np.array((1,0))}

def move_drone(pos,action):
    new_pos=pos+actions[action]
    if (0<=new_pos[0]<n) and (0<=new_pos[1]<n):
        return new_pos
    else:
        return pos
def get_reward(pos):
    return float(np.all(pos==goal_pos))

def get_neighboring_positions(pos):
    neighbors={}
    for action in actions.keys():
        neighbors[action]=move_drone(pos,action)
    return neighbors
```

```

def w(pos,x_eye):
    distance=np.linalg.norm(pos-x_eye)
    return np.exp(-distance**2/(2*sigma**2))

def is_intended_action(action,intent):
    return action==intent

def update_value_function():
    V=Value_function.copy()
    for x in range(n):
        for y in range(n):
            pos=np.array((x,y))
            storm_prob=w(pos,x_eye)
            neighbors=get_neighboring_positions(pos)
            value_update=-np.inf
            for intent in actions.keys():
                value_candidate=np.sum([(storm_prob/4 +
↪(1-storm_prob)*is_intended_action(a,intent))*(get_reward(neighbors[a]) +
↪gamma*V[tuple(neighbors[a])]) for a in actions.keys()])
                if value_candidate>value_update:
                    value_update=value_candidate
            Value_function[x,y]=value_update

```

```

[2]: # Value iteration until convergence
update_value_function()
loss=np.linalg.norm(Value_function-previous_value_function)
history=[loss]
while not loss<1e-4:
    previous_value_function=Value_function.copy()
    update_value_function()
    loss=np.linalg.norm(Value_function-previous_value_function)
    history.append(loss)
    print(f"Loss: {loss:.6f}")
plt.plot(history)
plt.yscale('log')
plt.xlabel('Iteration')
plt.ylabel('Loss (log scale)')
plt.title('Convergence of Value Function')
plt.grid()
plt.show()

```

```

Loss: 0.713082
Loss: 0.676943
Loss: 0.646419
Loss: 0.630601
Loss: 0.614890
Loss: 0.604435
Loss: 0.593896

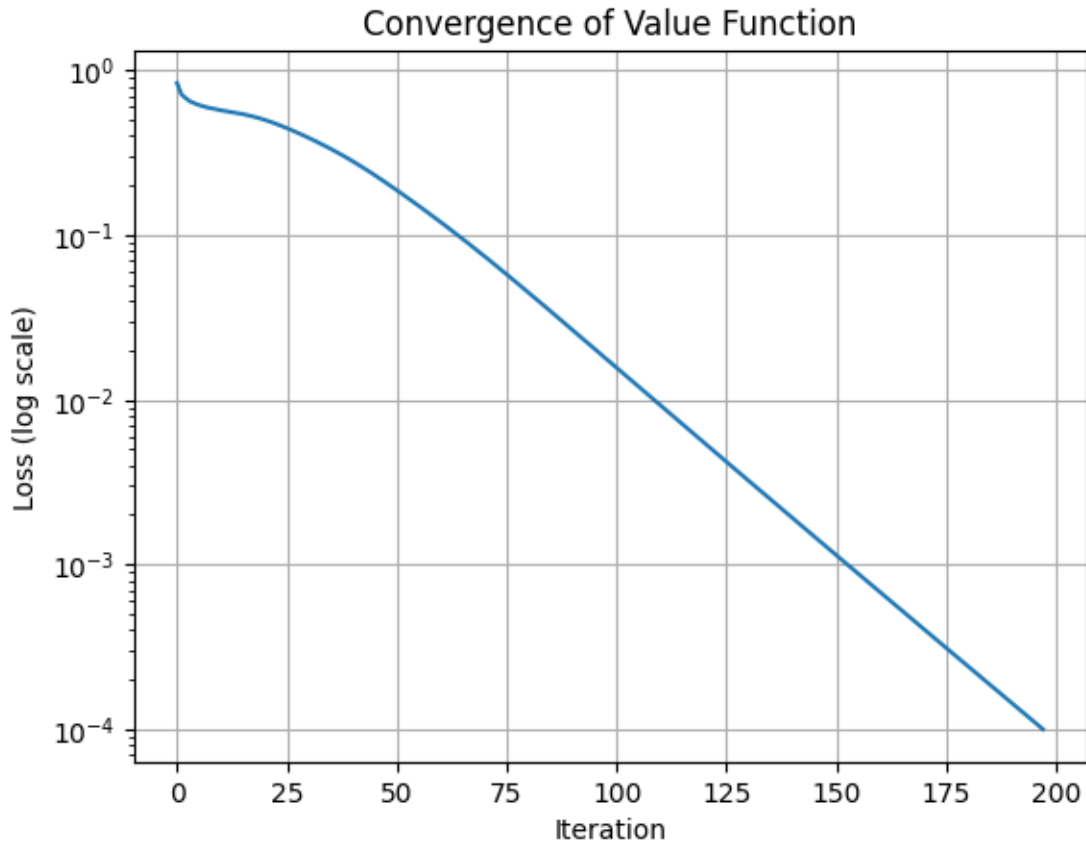
```

Loss: 0.585945
Loss: 0.578136
Loss: 0.571831
Loss: 0.565879
Loss: 0.560203
Loss: 0.554378
Loss: 0.548251
Loss: 0.541661
Loss: 0.534460
Loss: 0.526685
Loss: 0.518230
Loss: 0.509229
Loss: 0.499621
Loss: 0.489572
Loss: 0.479057
Loss: 0.468241
Loss: 0.457119
Loss: 0.445831
Loss: 0.434386
Loss: 0.422897
Loss: 0.411374
Loss: 0.399905
Loss: 0.388492
Loss: 0.377185
Loss: 0.365983
Loss: 0.354903
Loss: 0.343936
Loss: 0.333075
Loss: 0.322291
Loss: 0.311611
Loss: 0.301033
Loss: 0.290554
Loss: 0.280196
Loss: 0.269949
Loss: 0.259852
Loss: 0.249958
Loss: 0.240260
Loss: 0.230782
Loss: 0.221519
Loss: 0.212528
Loss: 0.203800
Loss: 0.195343
Loss: 0.187148
Loss: 0.179228
Loss: 0.171599
Loss: 0.164238
Loss: 0.157143
Loss: 0.150299

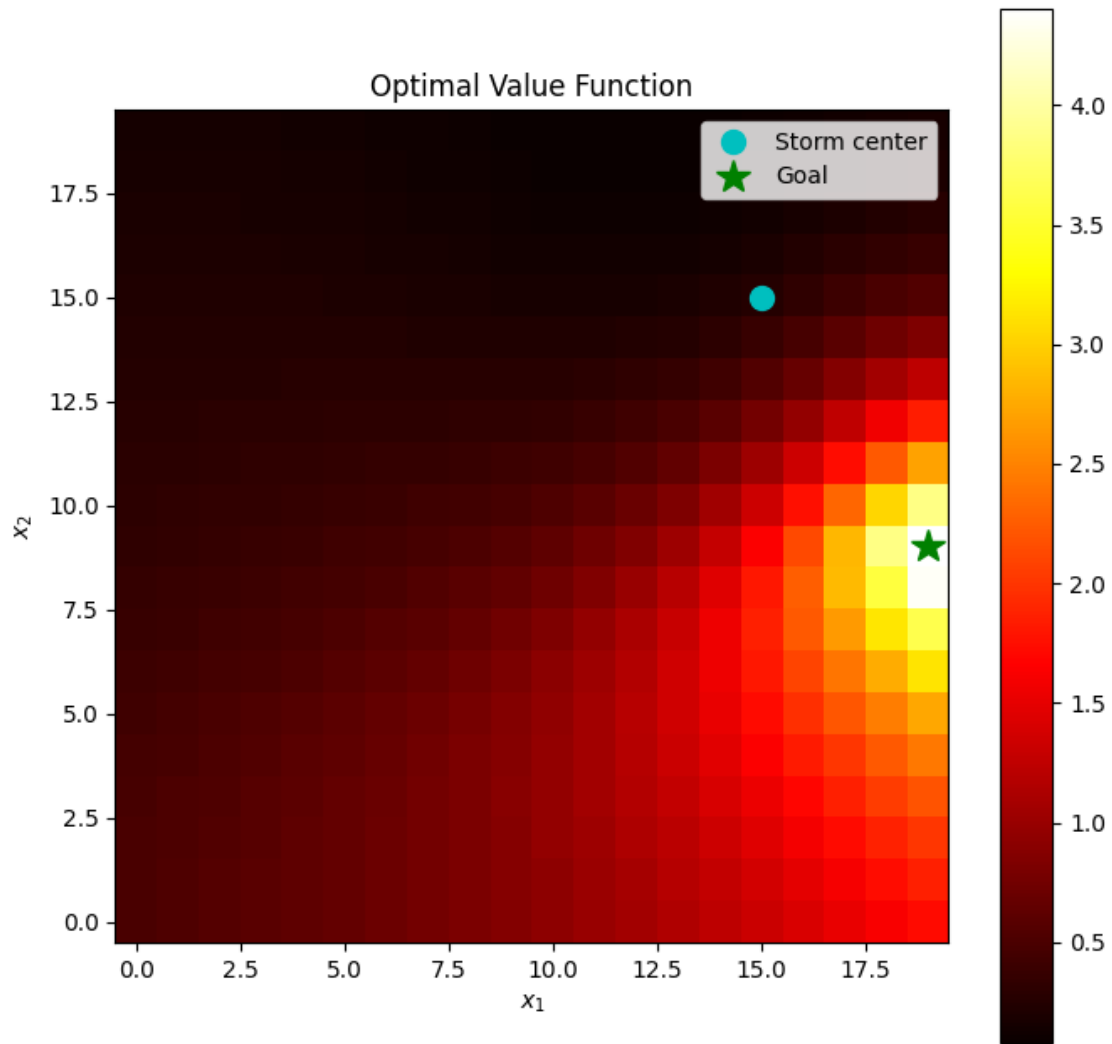
Loss: 0.143705
Loss: 0.137359
Loss: 0.131256
Loss: 0.125382
Loss: 0.119731
Loss: 0.114295
Loss: 0.109071
Loss: 0.104050
Loss: 0.099228
Loss: 0.094601
Loss: 0.090159
Loss: 0.085898
Loss: 0.081811
Loss: 0.077895
Loss: 0.074143
Loss: 0.070552
Loss: 0.067120
Loss: 0.063837
Loss: 0.060699
Loss: 0.057701
Loss: 0.054839
Loss: 0.052108
Loss: 0.049504
Loss: 0.047021
Loss: 0.044656
Loss: 0.042403
Loss: 0.040258
Loss: 0.038218
Loss: 0.036277
Loss: 0.034431
Loss: 0.032677
Loss: 0.031010
Loss: 0.029426
Loss: 0.027921
Loss: 0.026492
Loss: 0.025134
Loss: 0.023846
Loss: 0.022623
Loss: 0.021462
Loss: 0.020360
Loss: 0.019314
Loss: 0.018322
Loss: 0.017381
Loss: 0.016487
Loss: 0.015640
Loss: 0.014836
Loss: 0.014074
Loss: 0.013351

Loss: 0.012665
Loss: 0.012014
Loss: 0.011397
Loss: 0.010811
Loss: 0.010256
Loss: 0.009730
Loss: 0.009230
Loss: 0.008756
Loss: 0.008307
Loss: 0.007881
Loss: 0.007477
Loss: 0.007094
Loss: 0.006730
Loss: 0.006386
Loss: 0.006059
Loss: 0.005749
Loss: 0.005454
Loss: 0.005175
Loss: 0.004911
Loss: 0.004660
Loss: 0.004422
Loss: 0.004196
Loss: 0.003982
Loss: 0.003778
Loss: 0.003586
Loss: 0.003403
Loss: 0.003229
Loss: 0.003065
Loss: 0.002908
Loss: 0.002760
Loss: 0.002620
Loss: 0.002486
Loss: 0.002360
Loss: 0.002240
Loss: 0.002126
Loss: 0.002018
Loss: 0.001915
Loss: 0.001818
Loss: 0.001726
Loss: 0.001638
Loss: 0.001555
Loss: 0.001476
Loss: 0.001401
Loss: 0.001330
Loss: 0.001263
Loss: 0.001198
Loss: 0.001138
Loss: 0.001080

Loss: 0.001025
Loss: 0.000973
Loss: 0.000924
Loss: 0.000877
Loss: 0.000833
Loss: 0.000791
Loss: 0.000751
Loss: 0.000713
Loss: 0.000677
Loss: 0.000643
Loss: 0.000610
Loss: 0.000579
Loss: 0.000550
Loss: 0.000522
Loss: 0.000496
Loss: 0.000471
Loss: 0.000447
Loss: 0.000425
Loss: 0.000403
Loss: 0.000383
Loss: 0.000363
Loss: 0.000345
Loss: 0.000328
Loss: 0.000311
Loss: 0.000296
Loss: 0.000281
Loss: 0.000267
Loss: 0.000253
Loss: 0.000240
Loss: 0.000228
Loss: 0.000217
Loss: 0.000206
Loss: 0.000195
Loss: 0.000186
Loss: 0.000176
Loss: 0.000167
Loss: 0.000159
Loss: 0.000151
Loss: 0.000143
Loss: 0.000136
Loss: 0.000129
Loss: 0.000123
Loss: 0.000117
Loss: 0.000111
Loss: 0.000105
Loss: 0.000100



```
[3]: # Heatmap of optimal value function
fig, ax = plt.subplots(figsize=(8, 8))
im = ax.imshow(Value_function.T, origin='lower', cmap='hot')
ax.set_xlabel('$x_1$')
ax.set_ylabel('$x_2$')
ax.set_title('Optimal Value Function')
ax.plot(x_eye[0], x_eye[1], 'co', markersize=10, label='Storm center')
ax.plot(goal_pos[0], goal_pos[1], 'g*', markersize=15, label='Goal')
ax.legend()
plt.colorbar(im, ax=ax)
plt.show()
```



(b)

```
[4]: # (b) Compute optimal policy
action_names = list(actions.keys()) # ['up', 'down', 'left', 'right'] -> [0, 1, 2, 3]
policy = np.zeros((n, n), dtype=int)

for x in range(n):
    for y in range(n):
        pos = np.array((x, y))
        storm_prob = w(pos, x_eye)
        neighbors = get_neighboring_positions(pos)
        best_value = -np.inf
        for idx, intent in enumerate(action_names):
            value_candidate = sum(
```



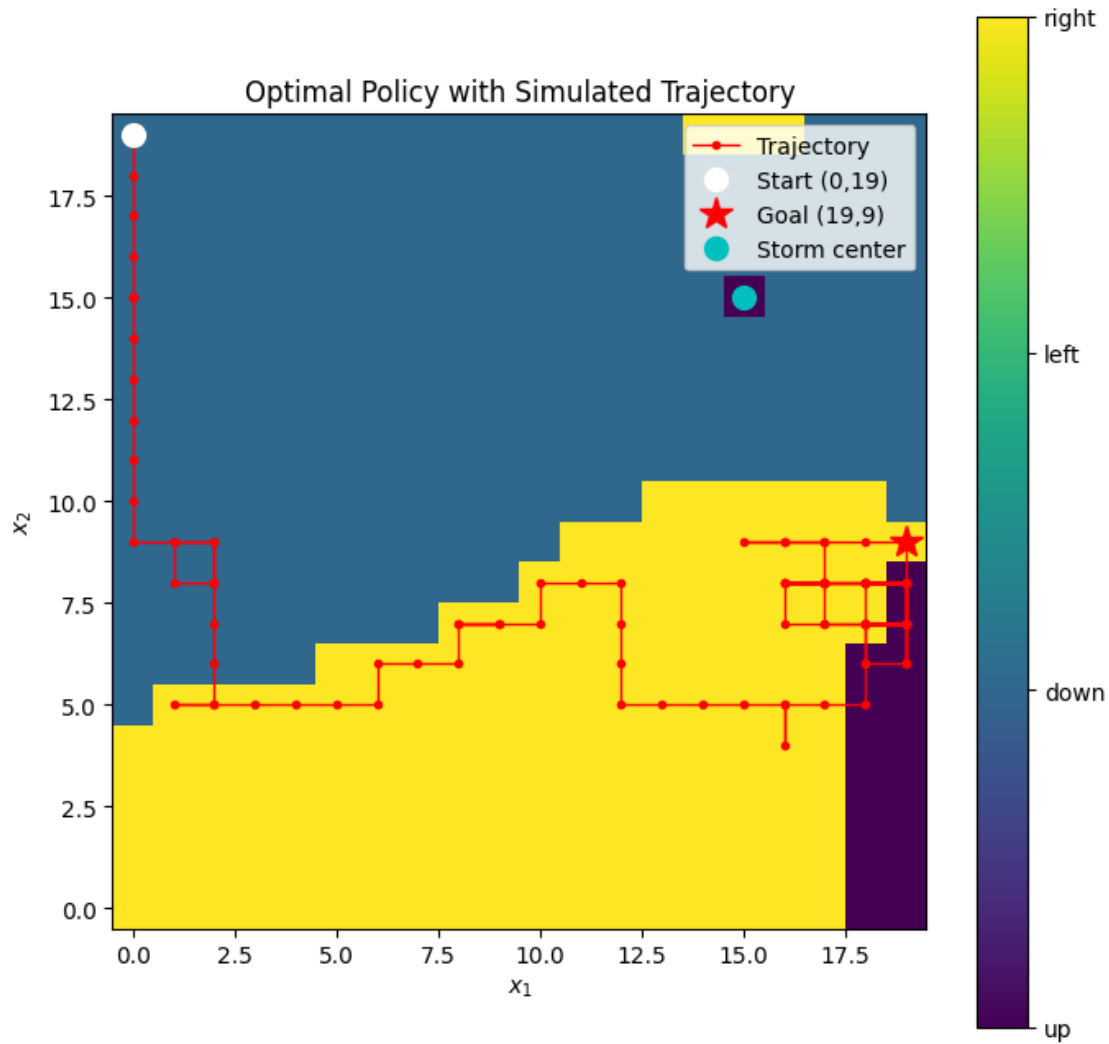
```

        (storm_prob/4 + (1 - storm_prob) * is_intended_action(a,
↪intent)) *
        (get_reward(neighbors[a]) + gamma *
↪Value_function[tuple(neighbors[a])])
        for a in action_names
    )
    if value_candidate > best_value:
        best_value = value_candidate
        policy[x, y] = idx

# Simulate from (0, 19) for 100 steps
np.random.seed(42)
pos = np.array([0, 19])
trajectory = [pos.copy()]
for _ in range(100):
    action = action_names[policy[pos[0], pos[1]]]
    omega = w(pos, x_eye)
    if np.random.rand() < omega:
        action = np.random.choice(action_names)
    pos = move_drone(pos, action)
    trajectory.append(pos.copy())
trajectory = np.array(trajectory)

# Plot policy heatmap with trajectory overlay
fig, ax = plt.subplots(figsize=(8, 8))
im = ax.imshow(policy.T, origin='lower', cmap='viridis')
ax.plot(trajectory[:, 0], trajectory[:, 1], 'r-o', markersize=3, linewidth=1,
↪label='Trajectory')
ax.plot(trajectory[0, 0], trajectory[0, 1], 'wo', markersize=10, label='Start,
↪(0,19)')
ax.plot(goal_pos[0], goal_pos[1], 'r*', markersize=15, label='Goal (19,9)')
ax.plot(x_eye[0], x_eye[1], 'co', markersize=10, label='Storm center')
ax.set_xlabel('$x_1$')
ax.set_ylabel('$x_2$')
ax.set_title('Optimal Policy with Simulated Trajectory')
cbar = plt.colorbar(im, ax=ax, ticks=[0, 1, 2, 3])
cbar.set_ticklabels(['up', 'down', 'left', 'right'])
ax.legend()
plt.show()

```



The policy steers the drone toward the goal at (19,9) while avoiding the storm center at (15,15). Far from the storm, the drone moves directly toward the goal. Near the storm, where the probability of being blown in a random direction is high, the policy routes the drone around the storm from below rather than going through it.

Problem2

May 22, 2026

0.1 3.2 Reach-avoid flight

(a)

We want to find $\mathbf{u}^* = \arg \min_{\mathbf{u}} \mathbf{p}^\top f(\mathbf{x}, \mathbf{u})$ where $\mathbf{p} = \nabla_{\mathbf{x}} V$ and the reduced dynamics are

$$f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \frac{v_y}{m} \frac{(T_1 + T_2) \cos \phi - C_D^v v_y}{\omega} - g \\ \frac{(T_2 - T_1) \ell - C_D^\phi \omega}{I_{yy}} \end{bmatrix}.$$

Expanding $\mathbf{p}^\top f$, only the terms involving T_1, T_2 matter for the minimization:

$$\mathbf{p}^\top f = \dots + T_1 \left(\frac{p_2 \cos \phi}{m} - \frac{p_4 \ell}{I_{yy}} \right) + T_2 \left(\frac{p_2 \cos \phi}{m} + \frac{p_4 \ell}{I_{yy}} \right).$$

This is linear in T_1 and T_2 (which are decoupled), so the minimum over $[0, T_{\max}]$ is attained at the extremes:

$$T_1^* = \begin{cases} T_{\max} & \text{if } \frac{p_2 \cos \phi}{m} - \frac{p_4 \ell}{I_{yy}} < 0, \\ 0 & \text{otherwise,} \end{cases} \quad T_2^* = \begin{cases} T_{\max} & \text{if } \frac{p_2 \cos \phi}{m} + \frac{p_4 \ell}{I_{yy}} < 0, \\ 0 & \text{otherwise.} \end{cases}$$

```
[1]: import jax.numpy as jnp
def optimal_control(self, state, grad_value):
    y, v_y, phi, omega = state
    p1, p2, p3, p4 = grad_value

    coeff_T1 = p2 * jnp.cos(phi) / self.m - p4 * self.l / self.Iyy
    coeff_T2 = p2 * jnp.cos(phi) / self.m + p4 * self.l / self.Iyy

    T_1 = jnp.where(coeff_T1 < 0, self.max_thrust_per_prop, 0.0)
    T_2 = jnp.where(coeff_T2 < 0, self.max_thrust_per_prop, 0.0)

    return jnp.array([T_1, T_2])
```

(b)

The target set is $\mathcal{T} = [3, 7] \times [-1, 1] \times [-\pi/12, \pi/12] \times [-1, 1]$. We need $h(\mathbf{x}) \leq 0 \iff \mathbf{x} \in \mathcal{T}$.

For a single interval $x \in [a, b]$, we have $(x - a)(x - b) \leq 0 \iff x \in [a, b]$. Taking the pointwise maximum:

$$h(\mathbf{x}) = \max\{(y - 3)(y - 7), (v_y - 1)(v_y + 1), (\phi - \frac{\pi}{12})(\phi + \frac{\pi}{12}), (\omega - 1)(\omega + 1)\}.$$

```
[2]: def target_set(state):
      y, v_y, phi, omega = state
      return jnp.max(jnp.array([
          (y - 3.0) * (y - 7.0),
          (v_y - 1.0) * (v_y + 1.0),
          (phi - jnp.pi / 12.0) * (phi + jnp.pi / 12.0),
          (omega - 1.0) * (omega + 1.0),
      ]))
```

(c)

The operational envelope is $\mathcal{E} = [1, 9] \times [-6, 6] \times [-\infty, \infty] \times [-8, 8]$. Similarly to part (b):

$$e(\mathbf{x}) = \max\{(y - 1)(y - 9), (v_y - 6)(v_y + 6), (\omega - 8)(\omega + 8)\}.$$

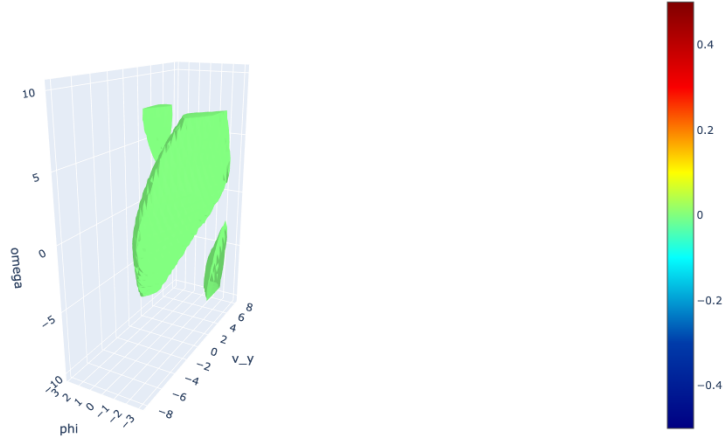
e does not contain ϕ as there is no constraint on ϕ .

```
[3]: def envelope_set(state):
      y, v_y, phi, omega = state
      return jnp.max(jnp.array([
          (y - 1.0) * (y - 9.0),
          (v_y - 6.0) * (v_y + 6.0),
          (omega - 8.0) * (omega + 8.0),
      ]))
```

(d)

```
[4]: from IPython.display import Image, display
      display(Image(filename='isosurface.png', width=700))
```

Zero isosurface, $y = 7.5000$



The isosurface exhibits a distinct diagonal tilt in the (ϕ, ω) plane. This shape is due to the physical coupling between the pitch angle and pitch rate under bounded control authority ($T_{\max}$). If the quadrotor has a positive ϕ (tilted right), it inherently requires a negative ω (rotating back left) to recover within the given time horizon, and vice versa. Therefore, states where ϕ and ω have opposite signs are far more recoverable because the existing angular velocity is already correcting the deviation. Conversely, states where they share the same sign require the actuators to first arrest the wrong rotation before straightening the drone, making them much harder to recover from. This coupling creates the diagonal ridge visible in the isosurface rather than an axis-aligned shape.

(e)

With DP/HJB we get a globally optimal policy over the whole state space, and at runtime it's just a table lookup so it's basically free to evaluate online. We also get the backward reachable set for free, which tells us exactly which states are recoverable. The main issue is the curse of dimensionality — the grid scales as $O(N^d)$ so both memory and offline compute blow up fast. That's why we had to drop from 6D to 4D here. Also, since the Hamiltonian is linear in the controls, the policy ends up being bang-bang ($T_1, T_2 \in \{0, T_{\max}\}$), which is optimal in theory but in practice means aggressive switching that's bad for actuators. And if we want to add obstacles later, we have to redo the whole computation.

MPC doesn't have the dimensionality problem since it just solves a local optimization at each step instead of gridding the state space. It's also easy to add new constraints or obstacles on the fly, and the optimization naturally gives smoother controls. The tradeoff is that it's only locally optimal, so it can fail from hard initial conditions where DP would have found a way out. Plus we need to solve an optimization at every timestep, which can be tight for something as fast as a self-righting maneuver.

Problem3

May 20, 2026

0.1 3.3 MPC feasibility

(a)

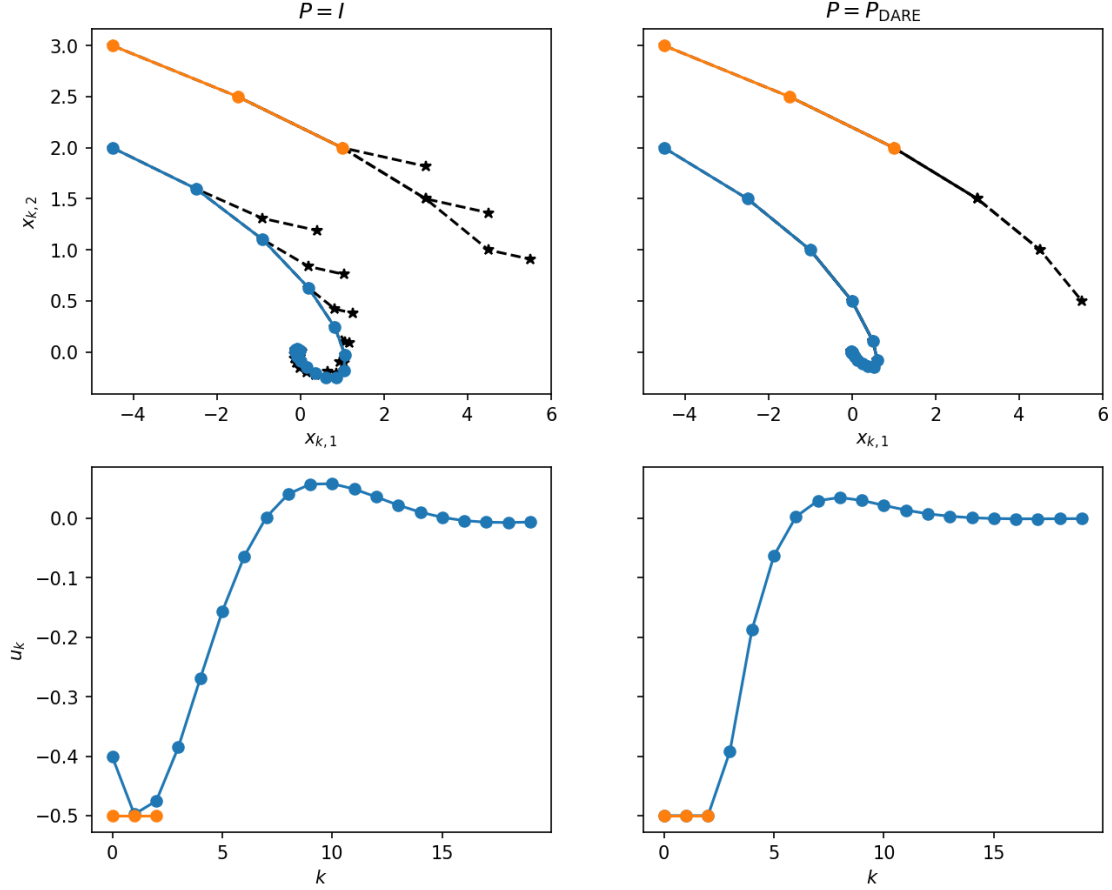
```
[1]: import cvxpy as cvx
def do_mpc(x0, A, B, P, Q, R, N, rx, ru, rf):
    n, m = Q.shape[0], R.shape[0]
    x_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

    cost = cvx.quad_form(x_cvx[N], P)
    for k in range(N):
        cost += cvx.quad_form(x_cvx[k], Q) + cvx.quad_form(u_cvx[k], R)

    constraints = [x_cvx[0] == x0]
    for k in range(N):
        constraints += [
            x_cvx[k + 1] == A @ x_cvx[k] + B @ u_cvx[k],
            cvx.norm_inf(x_cvx[k]) <= rx,
            cvx.norm_inf(u_cvx[k]) <= ru,
        ]
    constraints += [cvx.norm_inf(x_cvx[N]) <= rf]

    prob = cvx.Problem(cvx.Minimize(cost), constraints)
    prob.solve(cvx.CLARABEL)
    return x_cvx.value, u_cvx.value, prob.status
```

```
[2]: from IPython.display import Image, display
display(Image(filename='mpc_feasibility_sim.png', width=700))
```



For both values of P , the initial condition $x_0 = (-4.5, 2)$ (blue) converges to the origin. With $P = I$, convergence is slower and the predicted open-loop trajectories (dashed stars) diverge from the realized closed-loop trajectory. $P = I$ underestimates the true cost-to-go beyond the horizon, so the planner does not penalize the terminal state enough. With $P = P_{\text{DARE}}$, the predicted open-loop steps nearly coincide with the closed-loop trajectory (the stars overlap the dots), because P_{DARE} is the true infinite-horizon cost-to-go for the unconstrained LQR, so replanning produces essentially the same trajectory.

The initial condition $x_0 = (-4.5, 3)$ (orange) becomes infeasible at $t = 3$ for both values of P . The control saturates at $u = -0.5$ for all steps but the initial velocity $x_2 = 3$ is too large. The state drifts to $(3.0, 1.5)$ where MPC cannot find a feasible plan that satisfies $\|x_k\|_\infty \leq 5$ over the next $N = 3$ steps. This initial condition is outside the region of attraction for this MPC configuration regardless of the choice of P .

(b)

```
[3]: import numpy as np
def compute_roa(A, B, P, Q, R, N, rx, ru, rf, grid_dim=21, max_steps=20,
    ↪tol=1e-2):
    roa = np.zeros((grid_dim, grid_dim))
```

```

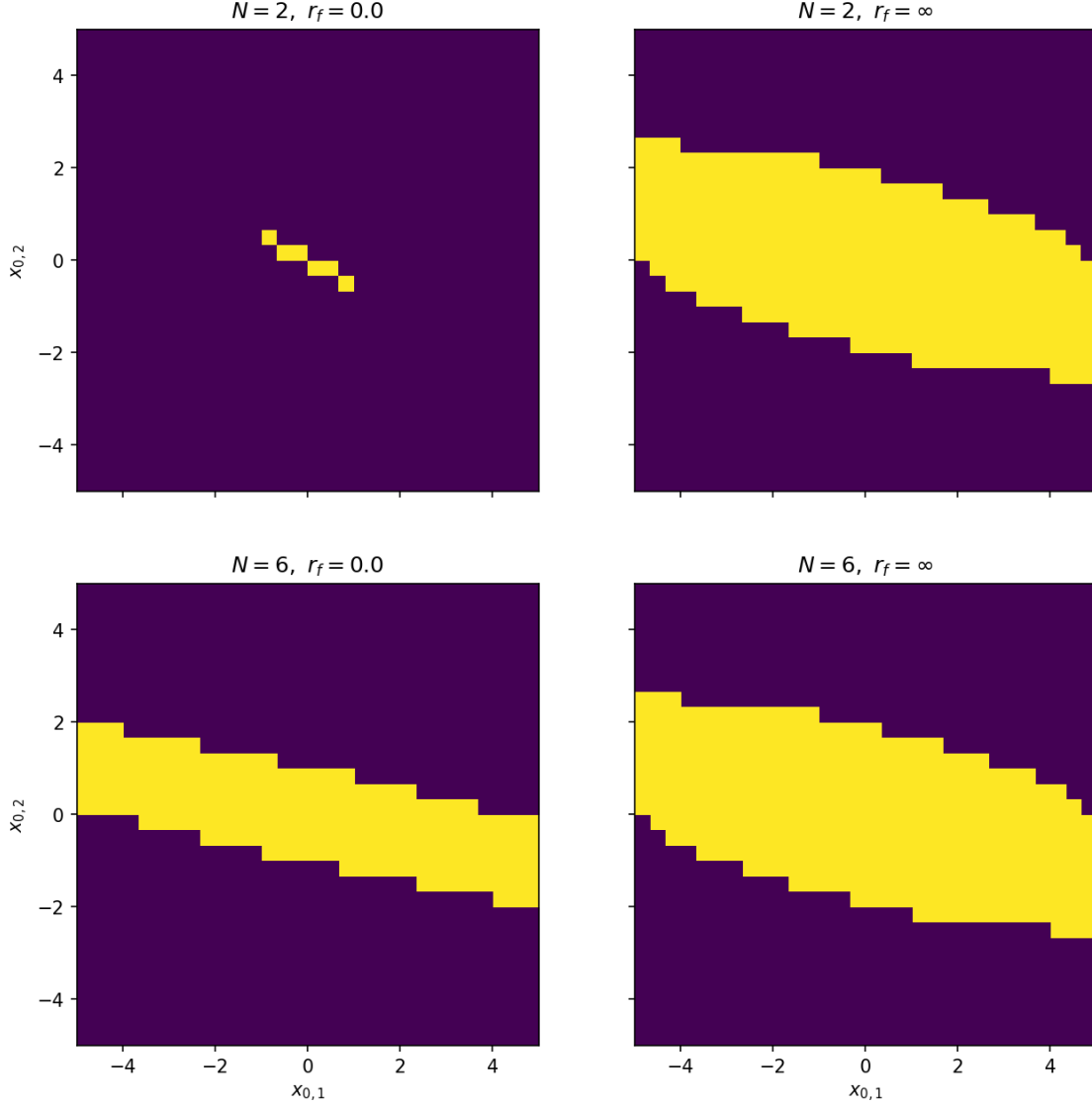
xs = np.linspace(-rx, rx, grid_dim)
for i, x1 in enumerate(xs):
    for j, x2 in enumerate(xs):
        x = np.array([x1, x2])
        for _ in range(max_steps):
            x_mpc, u_mpc, status = do_mpc(x, A, B, P, Q, R, N, rx, ru, rf)
            if status == "infeasible":
                break
            x = A @ x + B @ u_mpc[0, :]
            if np.linalg.norm(x) <= tol:
                roa[i, j] = 1
                break
return roa

```

```

[4]: from IPython.display import Image, display
display(Image(filename='mpc_feasibility_roa.png', width=700))

```

The ROA grows as we increase N and as we relax r_f .

With $r_f = 0$, the MPC must drive the state exactly to the origin within N steps. For $N = 2$ this is extremely restrictive: only a tiny set of initial states near the origin can reach $x = 0$ in 2 steps with $\|u\|_\infty \leq 0.5$, resulting in an almost empty ROA. Increasing to $N = 6$ gives the controller more steps to steer to the origin, so the ROA grows significantly.

With $r_f = \infty$, there is no terminal constraint. The MPC only needs to keep $\|x_k\|_\infty \leq r_x$ and $\|u_k\|_\infty \leq r_u$ over the horizon, which is a much weaker requirement. The ROA increases because more states admit a feasible plan: the controller no longer has to reach any specific terminal state within N steps, it just has to stay within the state constraints.

Increasing N also enlarges the ROA in both cases, since a longer horizon gives the controller more freedom to plan a feasible trajectory. However, the effect of r_f is more dramatic: going from $r_f = 0$

to $r_f = \infty$ at $N = 2$ produces a much larger jump in ROA than going from $N = 2$ to $N = 6$ at fixed r_f .

The diagonal shape of the ROA reflects the system dynamics: since x_1 represents position and x_2 velocity, states where x_1 and x_2 have the same sign (e.g., $x_1 > 0, x_2 > 0$) are harder to control back to the origin because the velocity pushes the position further away, while states where x_2 opposes x_1 are naturally returning.

Problem4

May 22, 2026

0.1 3.4 Terminal ingredients

(a)

$A = \begin{bmatrix} 0.9 & 0.6 \\ 0 & 0.8 \end{bmatrix}$ has eigenvalues 0.9 and 0.8, both strictly less than 1, so the uncontrolled system $x_{t+1} = Ax_t$ is asymptotically stable. This allows us to design $\mathcal{X}_T$ and P using only $x_{t+1} = Ax_t$ with $u = 0$.

Recursive feasibility requires $\mathcal{X}_T$ to be a positive invariant set for $x_{t+1} = Ax_t$ contained in $\mathcal{X} = \{x : \|x\|_2 \leq r_x\}$. We could take $\mathcal{X}_T = \mathcal{O}_\infty$, the maximal positive invariant set within $\mathcal{X}$.

To use the MPC stability theorem (for quadratic cost), we verify:

A0: $Q = Q^\top \succ 0$, $R = R^\top \succ 0$, $P \succ 0$. This holds since $Q = I$, $R = I$, and we choose $P \succ 0$.

A1: $\mathcal{X}$, $\mathcal{X}_T$, and $\mathcal{U}$ contain the origin in their interior and are closed. This holds by construction.

A2: $\mathcal{X}_T \subseteq \mathcal{X}$ is control invariant and bounded. Since $u = 0$ is a feasible control and $\mathcal{X}_T$ is positive invariant under $x_{t+1} = Ax_t$, it is control invariant for the full system.

A3: Since $u = 0$ is feasible and $Ax \in \mathcal{X}_T$ if $x \in \mathcal{X}_T$, assumption A3 becomes

$$-x^\top Px + x^\top Qx + x^\top A^\top PAx \leq 0, \quad \forall x \in \mathcal{X}_T,$$

which is true since, due to the fact that A is asymptotically stable,

$$\exists P \succ 0 \text{ s.t. } -P + Q + A^\top PA = 0 \quad (\text{discrete Lyapunov equation}).$$

(b)

Let $x \in \mathcal{X}_T$, i.e., $x^\top Wx \leq 1$.

Positive invariance: since $A^\top WA - W \preceq 0$,

$$(Ax)^\top W(Ax) = x^\top A^\top WAx \leq x^\top Wx \leq 1,$$

so $Ax \in \mathcal{X}_T$. Hence $\mathcal{X}_T$ is positive invariant for $x_{t+1} = Ax_t$.

State constraints: since $I - r_x^2 W \preceq 0$,

$$x^\top (I - r_x^2 W)x \leq 0$$

$$\Rightarrow \|x\|_2^2 \leq r_x^2 x^\top W x$$

Since $x \in \mathcal{X}_T$, $x^\top W x \leq 1$, so

$$\|x\|_2^2 \leq r_x^2 \cdot 1 = r_x^2$$

$$\Rightarrow \|x\|_2 \leq r_x$$

$$\Rightarrow \mathcal{X}_T \subseteq \mathcal{X} \quad (\text{respects state constraints}).$$

(c)

Let $M := W^{-1}$. Since W is symmetric positive definite, its eigenvalues $\lambda_i > 0$. The eigenvalues of $M = W^{-1}$ are $1/\lambda_i > 0$, and M is symmetric (the inverse of a symmetric matrix is symmetric), so M is also symmetric positive definite. Reciprocally, $M \succ 0 \Rightarrow W = M^{-1} \succ 0$ by the same argument.

Constraint 1: $A^\top W A - W \preceq 0$ with $W = M^{-1}$:

$$A^\top M^{-1} A \preceq M^{-1} \iff M^{-1} - A^\top M^{-1} A \succeq 0$$

Using hint 1 (pre/post multiply by $M \succ 0$):

$$M(M^{-1} - A^\top M^{-1} A)M = M - (AM)^\top M^{-1}(AM) \succeq 0$$

By Schur's complement lemma:

$$\begin{bmatrix} M & MA^\top \\ AM & M \end{bmatrix} \succeq 0$$

Constraint 2: $I - r_x^2 W \preceq 0$ with $W = M^{-1}$:

$$r_x^2 M^{-1} \succeq I \iff M^{-1} \succeq \frac{1}{r_x^2} I$$

By hint 2 ($A^{-1} \succeq \frac{1}{\gamma} I \iff A \preceq \gamma I$):

$$M \preceq r_x^2 I \iff r_x^2 I - M \succeq 0$$

Objective: $\log \det(W^{-1}) = \log \det(M)$, which is concave in $M \succ 0$.

The reformulated concave SDP is:

$$\begin{aligned} & \underset{M \succ 0}{\text{maximize}} && \log \det(M) \\ & \text{subject to} && \begin{bmatrix} M & MA^\top \\ AM & M \end{bmatrix} \succeq 0 \end{aligned}$$

$$r_x^2 I - M \succeq 0$$

(d)

```
[1]: import numpy as np
import cvxpy as cvx
import matplotlib.pyplot as plt

A = np.array([[0.9, 0.6], [0.0, 0.8]])
B = np.array([[0.0], [1.0]])
n = 2
rx = 5.0
ru = 1.0
Q = np.eye(n)
R = np.eye(1)
P = np.eye(n)
N = 4

# Solve the SDP for M
M = cvx.Variable((n, n), symmetric=True)
constraints = [
    cvx.bmat([[M, M @ A.T], [A @ M, M]]) >> 0,
    rx**2 * np.eye(n) - M >> 0,
]
prob = cvx.Problem(cvx.Maximize(cvx.log_det(M)), constraints)
prob.solve(cvx.SCS)

M_val = M.value
W_val = np.linalg.inv(M_val)
print(f"W = M^{{-1}} = \n{np.round(W_val, 3)}")

def generate_ellipsoid_points(M, num_points=100):
    L = np.linalg.cholesky(M)
    theta = np.linspace(0, 2 * np.pi, num_points)
    u = np.column_stack([np.cos(theta), np.sin(theta)])
    x = u @ L.T
    return x

X_pts = generate_ellipsoid_points(rx**2 * np.eye(n))
XT_pts = generate_ellipsoid_points(M_val)
AXT_pts = (A @ XT_pts.T).T

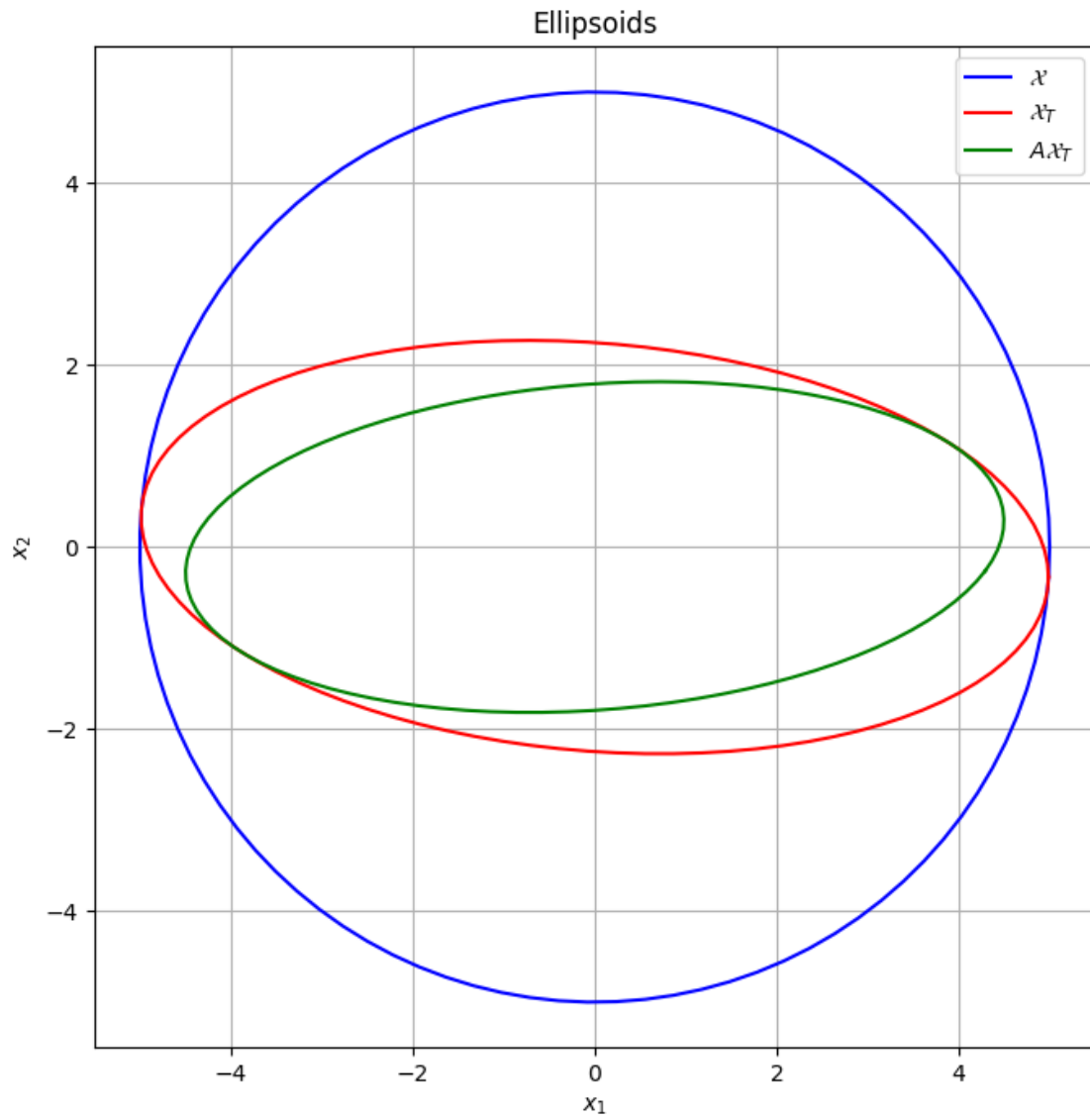
fig, ax = plt.subplots(figsize=(8, 8))
ax.plot(X_pts[:, 0], X_pts[:, 1], 'b-', label=r'$\mathcal{X}$')
ax.plot(XT_pts[:, 0], XT_pts[:, 1], 'r-', label=r'$\mathcal{X}_T$')
ax.plot(AXT_pts[:, 0], AXT_pts[:, 1], 'g-', label=r'$A\mathcal{X}_T$')
ax.set_xlabel(r'$x_1$')
ax.set_ylabel(r'$x_2$')
```

```

ax.set_title('Ellipsoids')
ax.set_aspect('equal')
ax.legend()
ax.grid(True)
plt.show()

```

$W = M^{-1} =$
 $\begin{bmatrix} 0.041 & 0.013 \\ 0.013 & 0.198 \end{bmatrix}$



(e)

```

[2]: # Parameterized MPC problem
x0_param = cvx.Parameter(n)
x_mpc = cvx.Variable((N + 1, n))
u_mpc = cvx.Variable((N, 1))

cost = cvx.quad_form(x_mpc[N], P)
for k in range(N):
    cost += cvx.quad_form(x_mpc[k], Q) + cvx.quad_form(u_mpc[k], R)

mpc_constraints = [x_mpc[0] == x0_param]
for k in range(N):
    mpc_constraints += [
        x_mpc[k + 1] == A @ x_mpc[k] + B @ u_mpc[k],
        cvx.norm2(x_mpc[k]) <= rx,
        cvx.norm2(u_mpc[k]) <= ru,
    ]
mpc_constraints += [cvx.quad_form(x_mpc[N], W_val) <= 1]

mpc_prob = cvx.Problem(cvx.Minimize(cost), mpc_constraints)

# Simulate from x0 = (0, -4.5) for 15 steps
T_sim = 15
x0 = np.array([0.0, -4.5])
x_actual = np.zeros((T_sim + 1, n))
u_actual = np.zeros((T_sim, 1))
x_planned = []

x_actual[0] = x0
for t in range(T_sim):
    x0_param.value = x_actual[t]
    mpc_prob.solve(cvx.CLARABEL)
    x_planned.append(x_mpc.value.copy())
    u_actual[t] = u_mpc.value[0, :]
    x_actual[t + 1] = A @ x_actual[t] + B.flatten() * u_actual[t, 0]

# State trajectories overlaid on ellipsoids
fig, ax = plt.subplots(figsize=(8, 8))
ax.plot(X_pts[:, 0], X_pts[:, 1], 'b-', label=r'$\mathcal{X}$')
ax.plot(XT_pts[:, 0], XT_pts[:, 1], 'r-', label=r'$\mathcal{X}_T$')
ax.plot(AXT_pts[:, 0], AXT_pts[:, 1], 'g-', label=r'$\mathcal{A}_T$')
for t in range(T_sim):
    ax.plot(x_planned[t][:, 0], x_planned[t][:, 1], '--*', color='k',
           markersize=3, linewidth=0.5)
ax.plot(x_actual[:, 0], x_actual[:, 1], '-o', color='tab:orange', markersize=5,
       label='Actual trajectory')
ax.plot(x_actual[0, 0], x_actual[0, 1], 'go', markersize=10, label=r'$x_0$')
ax.set_xlabel(r'$x_1$')

```

```

ax.set_ylabel(r'$x_2$')
ax.set_title('MPC closed-loop trajectory')
ax.set_aspect('equal')
ax.legend()
ax.grid(True)
plt.show()

# Control input over time
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(T_sim), u_actual[:, 0], '-o')
ax.set_xlabel(r'$t$')
ax.set_ylabel(r'$u_t$')
ax.set_title('Control input over time')
ax.grid(True)
plt.show()

```